Programmering og problemløsning, Aflevering 3

Carl-Emil Rohde

September 2022

Problemformuleringer

3i0

I første opgave, skal der skrives 4 forskellige funktioner, der på hver deres måde, tager en liste med talpar som input fx: [(10, 30); (-5, 127); (20, 90)], og returnerer en ny liste med turtle cmd's: [Turn 10; Move 30; Turn -5; Move 127; Turn 20; Move 90]. De 4 funktioner skal givet det samme input, returnere det samme output, men opnår som sagt resultatet på forskellige måder.

3i1

I denne opgave, skal der kodes to udvidelser til tegneprogramemt tree.fsx oplyst i opgaven. Den første udvidelse skal generere nogle tilfældige turtle commands hver eneste gang programmet køres, så træet skifter tilfældig position i canvas hver gang.

Den anden tilføjelse til programmet skal ved brug af en rekursiv funktion, et bestemt antal gange gentage processen med at tegne et tilfældigt træ på canvas, så der i stedet for kun ét træ, tegnes en lille skov af tilfældige træer på canvas.

Løsninger og overvejser

3i0

Den første funktion er en rekursiv funktion, der konverterer listen uden at bruge liste modulet. Her har jeg kaldt en rekursiv funktion "move", som tager et argument "n" som er en liste med talpar: "int*int" og returnerer en ny liste med turtle commands: "Canvas.turtleCmd list". Jeg matcher listen med et wildcard, hvorefter Turn og Move knyttes til hhv. hvert talpars første og andet tal. først listens hoved, og dernæst halen.

Nr 2 funktion er ikke rekursiv, men benytter i stedet List.map funktionen. Funktionen tager på samme måde som den første, en liste (Int*Int), som input, hvorefter den knytter Turn og Move til første og andet tal i et talpar. Dette gør den for alle talpar i listen.

Den tredje og fjerde funktion, bruger begge hhv. List.fold og list.foldBack til at tranformere listerne. Fold virker ved at opdaterer en akkumulator iterativt, ved at anvende i det her tilfælde "Move(y):: Turn(x)" på alle elementer i listen. list.foldback virker på nogenlunde samme måde, men baglæns. Figuren herunder, viser output i terminalen, når alle funktioner bliver givet det samme input:

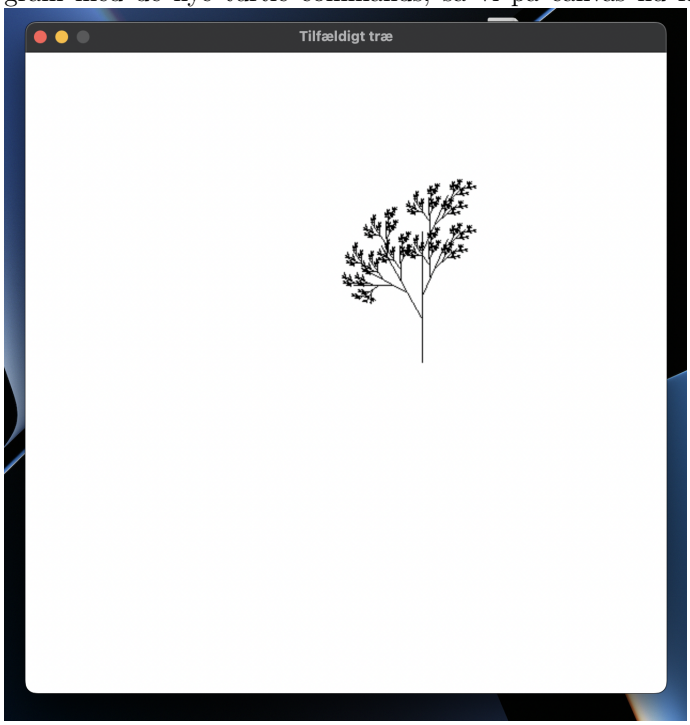
```
rohdee@MBPtilhCarlEmil src % dotnet fsi 3i0aF.fsx
  [Turn 10; Move 30; Turn -5; Move 127; Turn 20; Move 90]
  [Turn 10; Move 30; Turn -5; Move 127; Turn 20; Move 90]
  [Turn 10; Move 30; Turn -5; Move 127; Turn 20; Move 90]
  [Turn 10; Move 30; Turn -5; Move 127; Turn 20; Move 90]
rohdee@MBPtilhCarlEmil src %
```

3i1

For at løse første del af denne opgave, har jeg oprettet en funktion "RandomTree", der genererer to tilfældige heltalsværdier mellem 0 og 400, samt 0 og 250. Disse to værdier binder jeg til "a" og "b". Herefter opretter funktionen en ny liste med turtlecommands, som benytter værdierne "a" og "b". Således får vi altså nu nogle

optwidth
opt

nye tilfældige turtle commands, hver gang programmet køres. Det sidste funktionen gør, er at kalde Jon's tegneprogram med de nye turtle commands, så vi på canvas nu får tegnet et tilfældigt træ. som ses på figuren herunder:



For at løse anden del af opgaven, har jeg skrevet en rekursiv funktion, der tager to parametre "str" - størrelsen på træerne, og "x" - antallet af træer vi tegner, og derefter matcher x med først 0, i hvilket tilfælde intet vil blive tegnet på canvas og derefter et wildcard. når x rammer wildcardet, bliver der trukket -1 fra x, samtidig med at funktionen randomTree bliver kaldt. Denne proces køres igennem, indtil x matcher med 0. På den måde kan vi ved at sætte x til fx 10, bestemme hvor mange gange vi kalder tegnefunktionen og dermed hvor mange træer vi tegner. det færdige produkt ses på figuren herunder:

